

AlgoTraderSim

Real-Time Algorithmic Trading Simulator with Paper Trading & Strategy PnL Dashboards

Tech Stack: Python · WebSockets · Pandas · Plotly · Alpaca API

Version v1.0 · June 2026 · Internal Engineering Specification

Table of Contents

#	Section	Description
1	Project Overview	Goals, scope, personas, constraints
2	System Architecture	Services, data flow, WebSocket topology
3	Market Data Feed	Alpaca WebSocket integration, normalisation
4	Order Book Engine	Matching engine, order types, slippage model
5	Paper Trading Engine	Portfolio state, execution, fills simulation
6	Strategy Framework	Plugin interface, built-in strategies, backtest mode
7	PnL Dashboard	Plotly Dash layout, real-time updates, metrics
8	API & WebSocket Contracts	REST endpoints, WS message schemas
9	Persistence Layer	Time-series store, portfolio snapshots
10	Testing & Simulation Accuracy	Test harness, slippage regression

1. Project Overview

AlgoTraderSim is a high-fidelity paper-trading simulator that streams live market data via WebSockets, maintains a realistic order-book with slippage and fee modelling, and provides interactive Plotly dashboards for real-time strategy PnL monitoring.

Primary Language	Python 3.11+
Key Libraries	websockets 12, Pandas 2.2, Plotly/Dash 2.17, Alpaca-trade-api 3.3
Data Source	Alpaca Markets WebSocket (free paper-trading tier)
Target Users	Quant developers, algorithmic trading researchers
Deployment	Local Docker Compose / cloud VM (single-region)

1.1 Key Constraints

- Must process market data events at $\geq 10,000$ ticks/sec without event-loop blocking
- Order-book matching must be deterministic and replayable for backtesting
- Dashboard must update at ≤ 500 ms latency from fill event to screen refresh
- No real-money execution — all fills are simulated against live bid/ask
- Strategy plugins must be hot-loadable without simulator restart

2. System Architecture

Service	Role	Protocol
Market Data Gateway	Connects to Alpaca WS, normalises ticks, publishes to internal bus	WebSocket (upstream), asyncio queue (internal)
Order Book Engine	Maintains L2 book, matches orders, models slippage	asyncio internal
Paper Broker	Holds portfolio state, processes fills, tracks PnL	asyncio internal
Strategy Runner	Loads strategy plugins, feeds market events, collects signals	asyncio internal
REST API	Submit orders, query portfolio, configure strategies	HTTP/REST (FastAPI)
Dashboard Server	Real-time PnL/position Plotly Dash app	HTTP + WebSocket (Dash)
Time-Series DB	Stores tick data, fills, PnL snapshots	TimescaleDB / SQLite

3. Market Data Feed

3.1 Alpaca WebSocket Integration

```
// Alpaca WebSocket client
# market_data/alpaca_feed.py
class AlpacaFeed:
    WS_URL = 'wss://stream.data.alpaca.markets/v2/iex'
    async def run(self):
        async with websockets.connect(self.WS_URL) as ws:
            await self._authenticate(ws)
            await ws.send(json.dumps({'action': 'subscribe',
                                      'trades': self.symbols, 'quotes': self.symbols}))
            async for raw in ws:
                msgs = json.loads(raw)
                for msg in msgs:
                    await self._dispatch(msg)
```

3.2 Tick Normalisation

All upstream data normalised to a canonical **Tick** dataclass before publishing to the internal event bus:

```
// Canonical tick dataclass
@dataclass(slots=True)
class Tick:
    symbol: str
    timestamp: datetime
    price: Decimal
    size: int
    side: Literal['buy', 'sell', 'unknown']
    bid: Decimal | None
    ask: Decimal | None
    spread: Decimal | None # ask - bid
```

4. Order Book Engine

4.1 Supported Order Types

- **Market Order:** filled immediately at best available ask/bid $\pm$ slippage
- **Limit Order:** rests on book until price crosses limit level
- **Stop Order:** converts to market when trigger price touched
- **Stop-Limit:** converts to limit at stop trigger
- **Trailing Stop:** stop price tracks market by fixed offset or percentage

4.2 Slippage Model

Slippage is modelled as a function of **order size relative to average daily volume (ADV)** and **bid-ask spread**:

```
// Slippage estimation (square-root market impact model)
def estimate_slippage(order_qty: int, adv: int, spread: Decimal,
    impact_factor: float = 0.1) -> Decimal:
    participation = order_qty / adv
    market_impact = impact_factor * spread * Decimal(str(participation ** 0.5))
    half_spread = spread / 2
    return half_spread + market_impact
```

5. Paper Trading Engine

5.1 Portfolio State

Portfolio state is held in-memory and snapshotted to TimescaleDB every 30 seconds:

```
// Portfolio dataclass
@dataclass
class Portfolio:
    cash: Decimal
    positions: dict[str, Position] # symbol -> Position
    open_orders: dict[str, Order]
    fills: list[Fill]
    realized_pnl: Decimal = Decimal('0')
    @property
    def unrealized_pnl(self) -> Decimal:
        return sum(p.unrealized_pnl for p in self.positions.values())
    @property
    def total_equity(self) -> Decimal:
        return self.cash + sum(p.market_value for p in self.positions.values())
```

5.2 Risk Controls

- **Max position size:** configurable % of total equity per symbol (default 10%)
- **Max drawdown kill-switch:** halts all strategy signals if portfolio drawdown exceeds threshold (default 5%)
- **PDT protection:** optional flag to track and block > 3 round-trips in 5-day window
- **Sector concentration limit:** optional cap on gross exposure per GICS sector

6. Strategy Framework

6.1 Plugin Interface

```
// Strategy plugin base class
from abc import ABC, abstractmethod

class BaseStrategy(ABC):
    name: str
    symbols: list[str]

    @abstractmethod
    async def on_tick(self, tick: Tick) -> list[OrderRequest]: ...
    async def on_fill(self, fill: Fill) -> None: ... # optional hook
    async def on_day_open(self) -> None: ... # optional hook
    async def on_day_close(self) -> None: ... # optional hook
```

6.2 Built-in Strategies (Reference Implementations)

Strategy	Logic	Parameters
VWAP Mean-Reversion	Buy below VWAP - 1σ , sell above VWAP + 1σ	lookback_mins, sigma_threshold
Momentum Breakout	Long on N-day high breakout with volume confirm	breakout_days, vol_multiplier
Pairs Trading	Cointegration-based L/S on correlated pair	pair, z_entry, z_exit, lookback
Opening Range Breakout	Trade breakout from first 30-min range	range_mins, stop_pct, target_pct

7. PnL Dashboard

The Plotly Dash application provides a single-page real-time dashboard. Updates are pushed via Dash's **dcc.Interval** (200 ms) polling the internal state API.

7.1 Dashboard Panels

- **Equity Curve:** cumulative portfolio value vs SPY benchmark (line chart, intraday)
- **Position Table:** live positions with market value, unrealised PnL, %change
- **Order Blotter:** all open and recently filled orders with status
- **Trade Log:** fill history with price, size, slippage, commission
- **Strategy Metrics:** per-strategy realised PnL, win rate, avg win/loss
- **Risk Monitor:** gross exposure, net exposure, max drawdown, PDT counter
- **Tick Chart:** real-time candlestick with VWAP overlay for selected symbol

8. API & WebSocket Contracts

Method	Endpoint	Description
POST	/orders	Submit paper order (market/limit/stop)
DELETE	/orders/{order_id}	Cancel open order
GET	/portfolio	Full portfolio state snapshot
GET	/positions	All open positions
GET	/fills	Fill history with optional date filter
GET	/strategies	List loaded strategies and their status
POST	/strategies/{name}/toggle	Enable/disable strategy signal emission
WS	/ws/feeds	Server-push: fills, position updates, alerts

9. Persistence Layer

9.1 TimescaleDB Hypertables

```
// TimescaleDB DDL
-- tick_data (hypertable, partitioned by time)
CREATE TABLE tick_data (time TIMESTAMPTZ NOT NULL, symbol TEXT,
price NUMERIC, size INT, bid NUMERIC, ask NUMERIC);
SELECT create_hypertable('tick_data', 'time');

-- fills
CREATE TABLE fills (id UUID PRIMARY KEY, order_id UUID, symbol TEXT,
filled_at TIMESTAMPTZ, side TEXT, qty INT, fill_price NUMERIC,
slippage NUMERIC, commission NUMERIC, strategy TEXT);

-- portfolio_snapshots (every 30s)
CREATE TABLE portfolio_snapshots (snapshot_at TIMESTAMPTZ NOT NULL,
cash NUMERIC, equity NUMERIC, unrealized_pnl NUMERIC, realized_pnl NUMERIC);
SELECT create_hypertable('portfolio_snapshots', 'snapshot_at');
```

10. Testing & Simulation Accuracy

10.1 Test Harness

- **Replay mode:** historical tick data fed at configurable speed multiplier (1x–1000x) through identical code path as live mode
- **Determinism check:** two replay runs with identical seed must produce byte-identical fills
- **Slippage regression:** 10 reference trades replayed; slippage estimates must be within $\pm 5\%$ of baseline
- **Latency profiling:** end-to-end tick→signal→order→fill latency traced with asyncio instrumentation; p99 must be < 5 ms (excluding network)

10.2 Key Test Scenarios

- Market open / close boundary conditions (pre/post-market tick handling)
- Order book edge cases: zero-spread, locked market, halted security
- Strategy crash isolation — one bad plugin must not take down simulator
- Portfolio arithmetic: verify equity = cash + sum(position.market_value) at all times
- Drawdown kill-switch: confirm no orders submitted after threshold breach